NovaEventos: Chatbot de Reservas con FastAPI y SQLite

Sistema conversacional completo para cotizar salones, verificar disponibilidad por bloques de duración, agendar, modificar y cancelar reservas persistidas en SQLite.

1. Identificación del Proyecto

- Proyecto: NovaEventos (Versión 2.0 - Segundo Corte).
- Integrante: Santiago Zuluaga.
- Interfaz de usuario: Gradio (app_gradio.py - Puerto 7860).
- Agente y memoria: Python + Groq LLM (consulta_llm.py).
- API de negocio: FastAPI (main.py - Puerto 8011).
- Base de datos: SQLite transaccional (almacen_sqlite.py / reservas.db).

2. El Negocio y el Problema que Resuelve

- Negocio: Centro de convenciones con 4 espacios especializados para bodas, conferencias, cumpleaños y talleres ejecutivos.
- Problema previo: Cruces manuales de horarios, falta de cálculo automático de la hora de finalización y pérdida de contexto entre turnos.
- Solución: Validación matemática de intervalos en SQLite y confirmación obligatoria antes de guardar.

Este documento explica el funcionamiento completo de NovaEventos para cualquier lector, sin requerir haber asistido a las sesiones del curso.

1. SOLUCIÓN Y ALCANCE

Catálogo del Negocio, Usuarios, Tareas y Límites

Usuarios del sistema: Clientes organizadores de eventos (consulta y reservas en el chat) y coordinadores comerciales (control de agenda en SQLite). Horario de atención: 08:00 a 22:00.

Código ID	Servicio / Evento	Espacio Físico	Duración	Capacidad Máx.	Precio Base
EVT-BODA	Recepción de Boda / Gala Social	Salón Imperial	5 horas	200 personas	$4.500.000 COP
EVT-CORP	Conferencia Corporativa	Auditorio Nova	4 horas	120 personas	$2.800.000 COP
EVT-CUMP	Cumpleaños / Aniversario	Terraza Cristal	4 horas	80 personas	$2.200.000 COP
EVT-WORK	Taller Ejecutivo / Workshop	Sala Innovación	2 horas	40 personas	$950.000 COP

> Alcance y límites: Soporta conversación en varios turnos, sesiones independientes, confirmación/descarte/reinicio y CRUD sobre SQLite. No incluye pasarela de cobros en línea ni fraccionamiento de horas sueltas.

Relación entre Gradio, Python, el Modelo LLM, FastAPI y SQLite

Función de cada herramienta en el proyecto

- Gradio (app_gradio.py): Presenta la ventana de chat, el selector de sesiones (Sesion_1, Sesion_2, Sesion_3), los botones de confirmación/reinicio y la tabla SQLite en vivo.
- Python + LLM (consulta_llm.py): Construye el prompt con el catálogo, envía el historial a Groq (llama-3.3-70b-versatile) y extrae un JSON con los datos aportados.
- FastAPI (main.py): Recibe peticiones HTTP (GET, POST, PUT, DELETE) y aplica las reglas del negocio.
- SQLite (almacen_sqlite.py): Conserva las reservas en reservas.db.

Flujo de una solicitud de principio a fin

```
[Persona en Navegador Web]
        |
        v
[app_gradio.py] (Interfaz en Puerto 7860)
        |
        v
[consulta_llm.py] (Estado por Sesion)
   |-- 1. Consulta HTTPS JSON a Groq API
   |-- 2. Acumula borrador entre turnos
   +-- 3. Pide confirmacion al usuario
        |
        v  (Solo cuando el usuario confirma)
[main.py] (FastAPI en Puerto 8011)
   |-- GET    /disponibilidad
   |-- POST   /reservas
   |-- PUT    /reservas/{codigo}
   +-- DELETE /reservas/{codigo}
        |
        v
[almacen_sqlite.py] ---> [reservas.db]
```

Separación de responsabilidades: El chat nunca escribe directo en la base de datos; toda operación pasa por la API de FastAPI.

3. RECORRIDO DE USO COMPLETO

Ejemplo Paso a Paso: Solicitud Multi-Turno, Confirmación y Registro

Diálogo en varios turnos (conservando contexto)

- Turno 1 (Cliente): 'Quiero reservar una Conferencia Corporativa para el 2026-12-01' -> El bot registra EVT-CORP y la fecha, y pide hora, asistentes y contacto.
- Turno 2 (Cliente): 'A las 10:00 para 60 asistentes' -> El bot conserva el salón y la fecha anteriores, suma hora_inicio=10:00 y asistentes=60, y pide nombre y teléfono.
- Turno 3 (Cliente): 'Me llamo Santiago Zuluaga y mi contacto es 3005551122' -> El bot muestra el resumen completo y activa esperando_confirmacion = True.
- Turno 4 (Cliente): 'Sí, confirmo' -> Se envía POST /reservas a FastAPI.

Fila guardada en SQLite (reservas.db)

- codigo: NOVA-1004 (generado automáticamente)
- cliente_nombre: Santiago Zuluaga
- cliente_contacto: 3005551122
- servicio_id: EVT-CORP (Conferencia Corporativa)
- espacio: Auditorio Nova
- fecha: 2026-12-01
- hora_inicio: 10:00 | duracion_horas: 4 | hora_fin: 14:00
- asistentes: 60 (validado <= 120 máx.)
- estado: activa

Con el código devuelto (NOVA-1004), la persona puede consultar su reserva, cambiarla de fecha/hora o cancelarla en cualquier momento.

Horario de Atención, Duración por Servicio y Prevención de Cruces

Controles aplicados por almacen_sqlite.py

- 1. Cálculo de hora_fin según catálogo: Se convierte hora_inicio a minutos y se suma duracion_horas * 60.
- 2. Horario de operación (08:00 a 22:00): Si una Boda (5h) inicia a las 19:00, terminaría a las 24:00 y se rechaza.
- 3. Capacidad máxima por salón: Rechaza solicitudes que superen el aforo del espacio.
- 4. Rechazo de intervalos que se cruzan: Verifica reservas con estado = 'activa' en la misma fecha y salón.
- 5. Transacción BEGIN IMMEDIATE: Impide que dos solicitudes simultáneas ocupen el mismo bloque.

Verificación de cruces en SQLite

```
# 1. Calculo del intervalo solicitado
inicio_nuevo, fin_nuevo, hora_fin = calcular_intervalo(
    hora_inicio, servicio['duracion_horas']
)

# 2. Consulta de reservas activas en ese salon y fecha
SELECT codigo, hora_inicio, hora_fin
FROM reservas
WHERE fecha = ? AND espacio = ? AND estado = 'activa'

# 3. Condicion matematica de traslape [A, B) vs [C, D)
for fila in filas:
    inicio_exist = _a_minutos(fila['hora_inicio'])
    fin_exist = _a_minutos(fila['hora_fin'])
    if inicio_nuevo < fin_exist and fin_nuevo > inicio_exist:
        raise ValueError(
            f"Ocupado por reserva activa {fila['codigo']}"
        )
```

Manejo de cancelaciones: Al cancelar una reserva su estado pasa a 'cancelada'. La fila permanece en SQLite como historial pero libera el horario.

5. PRUEBAS Y RESULTADOS DOCUMENTADOS

Evidencias de los 6 Escenarios Evaluados en el Segundo Corte

Caso de Prueba	Entrada / Acción Ejecutada	Resultado Comprobado en Chat y SQLite	Evidencia
1. Varios turnos	Reserva en 4 mensajes separados para el 2026-12-01 a las 10:00.	Conserva datos previos y crea la fila NOVA-1004 (10:00-14:00).	evidencia_01_multiturno.png
2. Dos sesiones y reinicio	Boda en Sesion_1, Taller en Sesion_2 y reinicio de Sesion_1.	Separa conversaciones; Sesion_1 queda limpia y Sesion_2 intacta.	evidencia_02_sesiones_reinicio.png
3. Crear, modificar, cancelar	Modificar NOVA-1004 a las 15:00 (PUT) y luego cancelarla (DELETE).	Actualiza horario a 15:00-19:00 y cambia estado a 'cancelada'.	evidencia_03_crud_sqlite.png
4. Intervalos ocupados	Agendar en Auditorio Nova el 2026-11-15 a las 11:00.	Rechaza con HTTP 409 por cruce con NOVA-1001 (09:00-13:00).	evidencia_04_cruce_rechazado.png
5. Cancelación y persistencia	Cancelar NOVA-1001, reiniciar FastAPI y reservar a las 11:00.	Libera disponibilidad, crea NOVA-1005 y conserva las filas.	evidencia_05_cancelacion_persistencia.png
6. Manejo de errores	Solicitar Boda (5h) a las 19:00 (termina 24:00 > cierre 22:00).	Informa el problema y permite corregir la hora en la misma sesión.	evidencia_06_manejo_errores.png

Todas las pruebas cuentan con verificación automatizada en test_sistema_eventos.py (pytest) y capturas en docs/evidencias/.

6. EVOLUCIÓN DEL PROYECTO

Cambios frente a la Versión con Flowise y Archivos JSON

Dimensión	Versión Anterior (Flowise + JSON)	Versión Segundo Corte (Python + Gradio + FastAPI + SQLite)
Consumo del LLM	Nodos visuales cerrados en Flowise.	Petición HTTP directa en consulta_llm.py con headers, payload JSON y respaldo local.
Estado y Sesiones	Memoria implícita difícil de depurar o reiniciar.	Diccionario SESIONES[session_id] con historial, borrador y confirmación explícita.
Persistencia	Archivo plano reservas.json sin transacciones.	Tabla relacional reservas en SQLite (reservas.db) con semillas de prueba y transacciones.
Reglas de horario	Sin validación matemática de duración ni cruces.	Cálculo exacto de hora_fin y rechazo automático HTTP 409 ante traslapes.
Interfaz gráfica	Chat básico sin vista del estado ni de la base.	Gradio con selector de sesiones, visor de estado JSON y tabla SQLite en tiempo real.

> Responsabilidad asumida en código: El equipo programó directamente el control de sesiones, la validación de cruces y la persistencia SQL.

Conclusiones, Mejoras Pendientes y Estructura Entregada

Limitaciones Actuales y Mejoras Futuras

- Limitación 1: Duración fija por paquete según catalogo_eventos.json (2h, 4h o 5h).
- Limitación 2: El borrador no confirmado vive en memoria RAM hasta que el usuario confirma y pasa a SQLite.
- Mejora pendiente 1: Permitir contratar horas extra sobre la duración base de cada salón.
- Mejora pendiente 2: Envío de comprobante por correo electrónico al confirmar una reserva.

Documentos y Archivos del Repositorio

- README.md: Guía paso a paso de instalación y ejecución en dos terminales (FastAPI 8011 y Gradio 7860).
- docs/Bitacora_Segundo_Corte.md: Documentación técnica completa, evolución y pruebas enlazadas.
- docs/Presentacion_Segundo_Corte.pdf: Presentación autónoma del proyecto.
- docs/evidencias/: Capturas de pantalla y registro de pruebas.
- Código fuente: main.py, app_gradio.py, consulta_llm.py, almacen_sqlite.py y test_sistema_eventos.py.

Ejecución rápida: Terminal 1 -> uvicorn main:app --reload --port 8011 | Terminal 2 -> python app_gradio.py